

World Security

YC Application Strategy & Technical Truth Document — batch deadline July 27, 2026

Product: Autonomous penetration-testing agent with a persistent operational **World State** layer, built on the open-source **PentAGI** engine (MIT license).

Status of this document: every technical claim is verified against the codebase; every fact that cannot be verified is marked **[NEED FROM YOU]**. Nothing is invented.

Naming note (flag, not a debate): "World Security" ties to the World-State thesis, but is generic (hard to trademark, `.com` likely taken, weak for search). Keep a backup name. This document uses World Security throughout per your decision.

0 · The one thing that governs everything

Your YC story rests on two claims. The code says:

- 1. **The persistent World State is NOT yet wired into the agent loop.** The FSM validator, the DB tables, and the "World State" REST view are three disconnected pieces. The pentest agent never reads or writes the state tables.
 - 2. **DDQN / reinforcement learning does not exist in code — zero lines, anywhere.** It is a design concept only.
- Neither is fatal. But presented as *implemented*, a due-diligence reviewer finds the gap in ~15 minutes and loses trust in everything else. This plan makes claim #1 *true* before July 27 and never makes claim #2 at all.

1 · Technical Audit (verified against code)

1.0 Audit ledger — what exists / what doesn't (all 40 points)

Verdict key: **YES** exists & works · **PART** partial / not wired / view-only · **NO** does not exist. Every verdict is grep- or file-verified.

#	Question	Verdict	Evidence
1	What works end-to-end right now	YES	Autonomous multi-agent pentest engine runs (inherited PentAGI): flows → agents → Docker-sandboxed tools → results in UI
2	Your original code	YES	worldstate FSM+migration (untracked), <code>worldstate.go</code> , <code>attackpath.go</code> , <code>findings_extract.go</code> , <code>webpentest.go</code> , web-pentest migration, UI rebrand/tabs — all read-only views + data model
3	Inherited from PentAGI	YES	Entire engine: providers, tools, docker/terminal, queue, csum, graphiti client, auth, DB core, flows/tasks/subtasks, GraphQL, most frontend
4	Modified by you over PentAGI	PART	Mostly additive (new services + UI); little modification of core engine files
5	World State implementation location	PART	<code>backend/pkg/worldstate/lifecycle.go</code> + migration + <code>services/worldstate.go</code> — three disconnected pieces
6	Database migrations	YES	~20 goose migrations in <code>backend/migrations/sql/</code> ; <code>world_state</code> + <code>web_pentest</code> ones are yours
7	Entity schemas	PART	<code>world_state_entities</code> (typed, JSONB props) defined in migration — but no Go model/query uses it
8	State transitions (definition)	YES	<code>allowedTransitions</code> map + <code>ValidateTransition()</code> in <code>lifecycle.go</code> , unit-tested
9	Explicit lifecycle FSM	PART	Correct + tested, but untracked in git and imported by nothing
10	<code>world_state_query</code> tool	NO	No such tool in <code>pkg/tools/</code> ; grep empty

11	<code>world_state_update</code> tool	NO	Same — agent cannot mutate state
12	Planner queries state before action	NO	No state-query in planning path; planner is unmodified PentAGI
13	Executor updates state after tool exec	NO	Nothing writes <code>world_state_entities/_transitions</code>
14	Pre-execution gate	NO	No gate before command execution
15	Duplicate-action prevention	NO	No dedup logic in agent loop
16	Credential persistence	NO	No credential entity/table/store
17	Cross-subtask credential reuse	NO	Nothing to reuse from
18	Finding deduplication	NO	<code>findings_extract.go</code> re-extracts via LLM per request; no dedup/persistence
19	Authorization-as-state / scope enforcement	NO	"Authorizations" is a UI tab only; no scope entity checked before actions
20	Auditable transition history	PART	<code>world_state_transitions</code> table exists but nothing writes it
21	Evidence validation	NO	<code>evidence</code> JSONB column defined but never populated/validated
22	PostgreSQL integration	YES	Required; pgvector; GORM + goose (inherited)
23	Neo4j / Graphiti integration	YES	Optional; <code>pkg/graphiti/</code> client; used by worldstate/attackpath views
24	Operational state vs knowledge graph separation	NO	No operational state layer exists yet; only the Graphiti KG + log-derived view
25	Methodologies encoded in prompts/workflows	PART	Inherited PentAGI agent prompts + your web-pentest phase types (migration 20260419)
26	Security tools supported	YES	~15 (nmap, sqlmap, nuclei, ffuf, gobuster, hydra, metasploit...) via <code>pkg/tools/</code> + terminal
27	LLM providers supported	YES	11: anthropic, openai, gemini, bedrock, ollama, deepseek, glm, kimi, qwen, custom, embeddings
28	UI	YES	React/TS app; flows, settings, World State graph view, Authorizations tab
29	API	YES	REST (Gin) + GraphQL + Swagger
30	Multi-user support	YES	JWT/OAuth2/API tokens, roles/permissions (<code>auth.go</code> , <code>roles.go</code>)
31	Deployment mechanism	YES	Docker Compose (core + observability + langfuse + graphiti); installer TUI
32	Docker / Kubernetes	PART	Docker  ; no Kubernetes manifests
33	Tests	YES	94 Go <code>_test.go</code> files; worldstate FSM well-tested; attackpath test is trivial
34	Benchmark scripts	NO	No benchmark/eval/A-B harness anywhere
35	Telemetry	YES	OpenTelemetry + Langfuse (<code>pkg/observability/</code>)
36	Real execution logs (committed)	NO	No run logs/results committed to repo
37	Evidence of real A/B runs	NO	None exist
38	Measurable performance improvements	NO	No metrics of any kind
39	RL / DDQN code	NO	Zero matches repo-wide; only a DeepSeek model description mentions "reinforcement learning"
40	Production blockers	PART	World State un-wired + untracked; no metrics; no K8s; single-author fork; EULA/license consistency to confirm

One-line reading of the ledger: the *platform* (engine, tools, providers, API, UI, auth, deploy, telemetry, tests) mostly says YES — because it's inherited PentAGI. The *differentiated thesis* (state query/update, planner-reads, executor-writes, gate, dedup, credentials, auditable transitions, RL, metrics) mostly says NO — because it isn't built yet. That gap is exactly what the July plan closes.

1.1 The World State: three pieces that don't connect

Piece	File	What it is	Wired in?
FSM validator	backend/pkg/worldstate/lifecycle.go	Correct, unit-tested 110-line state machine: 7 states (unknown→discovered→scanning→assessed→vulnerable→exploited→remediated), ValidateTransition()	NO Untracked in git; imported only by its own test.
DB tables	migrations/...world_state_tables.sql	world_state_entities , _links , _transitions + lifecycle enum	NO Untracked; no Go code reads/writes them.
"World State" REST view	server/services/worldstate.go	Read-only UI graph endpoint GetWorldState()	STATELESS Wired to router, but see below.

What GetWorldState actually does: each request re-derives a graph from flows/tasks/subtasks , regex-scrapes the last 500 terminal logs for tool names/domains/URLs, and optionally reads Graphiti/Neo4j nodes. It **never reads the state tables** and **never uses the FSM**. version is hardcoded; updatedAt = time.Now() . A stateless visualizer that reverse-engineers a graph from logs.

1.2 The described agent loop does not exist yet

The cycle Query State → Select Action → Execute → Validate → Update State → Record Transition → Re-plan is **not implemented**:

- No world_state_query / world_state_update agent tool exists.
- Planner does not query state before acting; executor does not write state/transitions after acting.
- No pre-execution gate, no duplicate-action prevention.
- No credential entity/store; no cross-subtask credential reuse.
- findings_extract.go = on-demand LLM extraction, not a deduplicated persistent store.
- attackpath.go = paths computed on the fly from Graphiti at request time; not persisted; needs Neo4j.
- Authorization = a UI tab, not scope-as-state enforcement. The evidence column exists but nothing writes it.

1.3 DDQN / RL — explicit verdict: ABSENT

Repo-wide search for ddqn|dueling|double deep|q-network|q-learning|reinforcement|replay buffer|reward|policy network|epsilon-greedy|target network = **zero matches**. The only "reinforcement learning" string is a DeepSeek model description. No state/action space, reward, replay buffer, target network, training loop, inference path, policy, or results. **Never present as implemented or in-progress**. (Your public redscope.html and README contain zero RL/DDQN/world-state mentions — nothing public overclaims yet.)

1.4 What genuinely works (inherited from PentAGI, real)

- Autonomous multi-agent pentest engine (Researcher/Developer/Executor + support agents).
- 11 LLM providers; ~15 security tools + 6 search providers; Docker-sandboxed execution.
- REST + GraphQL API, React UI, multi-user auth (JWT/OAuth2/API tokens).
- PostgreSQL + pgvector (required); Neo4j/Graphiti (optional); Docker Compose deployment (no Kubernetes).
- 94 Go test files; OpenTelemetry + Langfuse. **No benchmark harness, no committed logs, no A/B results, no metrics.**

1.5 Provenance: yours vs PentAGI

Fork of vxcontrol/pentagi , **MIT-licensed** → commercial/closed derivatives allowed, no copyleft IP risk. **FLAG** Clarify whether the repo's EULA.md / NOTICE are inherited or yours, so the license story is consistent.

Your original additions: the worldstate FSM package + migration (untracked); read-only analytics/visualization services (worldstate, attackpath, findings_extract, webpentest); the web-pentest agent-types migration; UI rebrand + Authorizations tab + World State graph view + knowledge-graph view; the writeup + diagrams.

Inherited (largely unmodified): the entire autonomous engine — providers, tools, docker/terminal sandbox, queue, chain-summarization, graphiti client, auth, database core, flows/tasks/subtasks, GraphQL, most of the frontend.

Honest summary: today your contribution is read-only visualization/analytics endpoints + UI on top of PentAGI, plus an un-wired World-State data model. The autonomous capability itself is upstream. The plan makes the World State loop your real, working contribution.

2 · Product Truth Table

Key: **WIRED** implemented & wired · partial · built-but-not-wired · **ABSENT**

Claim	Status	Safe to claim in YC app?
Persistent World State	built, not wired	No — only as "designed"
Typed entities	partial	No (not persisted)
Explicit lifecycle FSM	built, not wired	Only as "designed + unit-tested, not integrated"
Auditable transitions	planned	No
Planner reads state before acting	ABSENT	No
Executor writes state after acting	ABSENT	No
Duplicate recon prevention	ABSENT	No
Credential persistence / reuse	ABSENT	No
Finding deduplication	absent / on-demand only	No
Attack-path persistence	partial (view only)	"visualization"  ; "persistence" No
Authorization scope as entity	ABSENT	No
Pre-execution safety gate	ABSENT	No
Methodology-driven autonomous pentest	partial (inherited)	 if framed as inherited + extended
Usable by non-expert users	partial, unverified	Soft claim only
Reinforcement learning / Dueling Double DQN	ABSENT	Never
Self-learning / policy improvement	ABSENT	Never
Lower dup-rate / tokens / higher coverage / faster finding	NO EVIDENCE	No (until A/B)

3 · Positioning decision

Product (10-sec): "An autonomous pentester that doesn't forget — it remembers what it already scanned, exploited, and which credentials it found, so it goes deeper instead of repeating itself."

Company (big story): the operational-state + audit layer that makes autonomous security agents reliable and trustworthy enough for enterprises.

Passes all five tests: understood in 10s, not a research project, not the 4th identical pentester (state = reliability = the gap others fight), credible market today (AppSec budgets), looks big (control-plane expansion).

Backup: pure "Stateful Autonomous Security" infra framing — deploy if partners say the pentest market is crowded. **Drop:** "World Models for Security" (research trap) and the word *learning*/RL (no RL exists).

4 · Competitor teardown

⚠ funding/batch marked [VERIFY] — confirm before submission.

Company	Autonomous	Continuous	Exploit	Remediation	Persistent op-state	Lifecycle FSM	Auditable transitions	RL policy
XBOW	✓	~	✓	×	×	×	×	×
MindFort (YC) [V]	✓	✓	✓	✓	×	×	×	×
RunSybil [V]	✓	~	✓	×	×	×	×	×
Ethiack	✓	✓	✓ PoE	~	×	×	×	×
Horizon3 / NodeZero	✓	✓	✓	✓	~	×	×	×
Pentera	~	✓	✓	✓	~	×	×	×
PentAGI (upstream)	✓	×	✓	~	×	×	×	×
World Security (if wired)	✓	roadmap	✓	~	building	built	building	×

Synthesis: the "persistent operational state + auditable transitions" lane is **open** — no competitor claims it. But it is **thin as a pure tech moat** (copyable in weeks). The defensible story leans on: (a) **audit/compliance for autonomous offense** — a signed, replayable record of everything the agent did, required before enterprises trust autonomous exploitation; and (b) **compounding per-customer environment models**. Neither exists yet; both are the roadmap. **Do not claim the FSM itself is the moat.**

5 · YC Application draft (standard 2026 form)

Per answer: **Answer** → why it works → ⚠ risk → 📈 stronger with evidence.

Company name

World Security (confirm; keep a backup name).

One-line (≤50 chars)

"Autonomous pentester that remembers what it did." → 📈 post-metrics: "Autonomous pentester that doesn't repeat itself."

What are you making?

"World Security is an autonomous penetration-testing agent that maintains a persistent operational model of the target — every host, service, credential, and finding as a typed entity with an enforced lifecycle. Before it acts it queries what it already knows; after each action it records what changed. This stops the repeated scans, forgotten credentials, and rediscovered findings that make today's autonomous pentest agents waste tokens and miss depth. It's built on the open-source PentAGI engine (MIT), with our operational-state and audit layer on top."

Why: product-first, names the pain, discloses the fork. ⚠ "before it acts it queries state" must be TRUE at submission (Steps 3–5). 📈 add A/B numbers.

How far along?

"Working platform (forked from PentAGI, MIT): autonomous multi-agent pentest, 11 LLM providers, Docker-sandboxed tools, REST/GraphQL, React UI, multi-user auth — running today. Our additions: a designed-and-tested lifecycle FSM + typed-entity schema,

live attack-path/findings visualization, web-pentest phase workflows. In progress this month: wiring the state layer into the agent loop + a baseline-vs-stateful benchmark."

⚠️ "forked" invites "what's actually yours" — answer head-on. 📌 "Benchmark complete: [numbers]. Demo: [link]."

Competitors / who you fear

"XBOW (best-funded, autonomous exploitation, tops HackerOne), MindFort (YC, continuous + remediation), RunSybil, Ethjack; incumbents Horizon3/Pentera. None publicly builds an explicit operational-state model with an auditable, enforced action lifecycle — our wedge. I fear XBOW most on raw capability; our bet is that reliability and auditability, not model horsepower, block enterprise trust in autonomous offense."

Remaining fields — need your facts

- **Why this idea / expertise / how you know people need it:** [NEED §7-H origin story + §7-G background]
- **How long working / full-time:** [NEED §7-C]
- **Tech stack:** Go (REST+GraphQL), React/TS, PostgreSQL+pgvector, optional Neo4j/Graphiti, Docker; 11 LLM providers; OTel + Langfuse; forked from PentAGI (MIT). ✅ verified
- **Users / revenue:** [NEED §7-D] — if none: "Pre-launch. Applying on working platform + insight + early benchmark."
- **How you make money:** per-target/per-seat SaaS for continuous autonomous web-app pentesting → audit/control layer for other security agents. [NEED §7-F buyer/pricing]
- **Legal entity / investment / fundraising:** [NEED §7-B]
- **Category:** Security (B2B SaaS)
- **Most impressive non-startup build:** [NEED §7-G], public/safe only
- **Founders known how long / how met:** [NEED §7-A] (skip if solo; be ready to defend solo)
- **"Hacked a non-computer system" / why apply to YC / how heard:** [NEED — personal]
- **Equity / IP / employer risk:** [NEED §7-A — flag any non-compete now]

6 · 90-second demo script

Two side-by-side runs, same target (OWASP Juice Shop), pre-recorded + one live fallback. Left = run log; right = World State graph.

Time	Screen	Narration	Notes
0:00–0:10	Flow-create	"Autonomous pentest agents forget what they've done. Same agent, same target, twice."	Hide branding.
0:10–0:30	Baseline, zoom 2 near-identical scans	"Without state: scans the host in subtask 1... scans it again in subtask 3. Watch the duplicate."	Red box on duplicate.
0:30–0:42	Baseline: found cred, later asks for creds again	"Found admin creds here — forgot them here."	Red boxes.
0:42–0:55	Stateful; right panel populates (host→scanning→assessed)	"Now with the World State layer. Before acting, it queries what it knows."	Zoom right panel.
0:55–1:10	Pre-exec gate: "target already scanned (entity #4), skipping"	"It sees the host is already scanned — and skips the duplicate. That's the FSM gate."	Most important frame. Green box.
1:10–1:22	Later subtask pulls stored credential	"It retrieves the credential it stored earlier and uses it in an authorized step — no rediscovery."	Green box + transition record.
1:22–1:30	Metrics table (from CSV)	"Same agent, same model: duplicate actions down X%, tokens/finding down Y%. It's the state, not the model."	Real numbers only.

Hide: logos, API keys, untracked-file mess, crashed runs. **Zoom:** the duplicate (baseline) + the skip/reuse (stateful). **Backup:** the pre-recorded split-screen — never run live in the submission.

7 · Founder questionnaire (private facts needed)

- **A. Founders:** legal name(s); solo/cofounders; roles; equity split; when/how met; how long together; who's full-time; who can relocate; IP owner; **employer-IP risk / non-compete**.
- **B. Company:** legal entity (country, date, name)? domain/product URL? repo public/private? prior investment/commitments/accelerator? fundraising now?
- **C. Product:** exact start date; total hours + full-time share; time to first pentest run for a non-expert.
- **D. Traction:** users? pilots/design partners? completed runs, targets, findings, confirmed vulns? anyone asked/willing to pay? (zero is a valid, honest answer.)
- **E. Metrics:** from the A/B (Jul 18–20). None exist until then — state none.
- **F. Customer:** who's the buyer (Head of AppSec / CISO / consultancy / dev lead)? talked to any? what did they say?
- **G. Founder-market fit:** AppSec, detection eng, AI-security, AWS/automotive security, research, IEEE/talks, open-source, production AI — tag each verified/public/confidential/safe-to-cite. Any accepted talk/paper to cite?
- **H. Origin story:** the concrete moment — which run, what repeated (which scan / forgotten credential / rediscovered finding), what you tried first, why vector memory didn't fix it, when you realized you needed explicit state, what you built first, prototype speed, what surprised you. **Your most persuasive asset**.

8 · Founder video — pick V1 (technical founder)

"I'm [name]. I build autonomous pentest agents, and I kept watching them fail the same way: scan a host, then scan it again two steps later. Find admin credentials, then forget them. The problem isn't the model — these agents have conversation history and vector search, but no operational model of the target. So I built one: every host, credential, and finding is a typed entity with an enforced lifecycle. Before the agent acts, it checks what it already knows. After it acts, it records what changed — an auditable trail of everything the autonomous attacker did. Same model, but it stops repeating itself and goes deeper. It's built on PentAGI, the open-source engine — our layer is the state and audit on top. [If wired:] Here it is skipping a duplicate scan, live."

Backup V2 (customer problem) if you land a buyer quote **[NEED \$7-F]**. Avoid V3 (ambitious thesis) as primary — research-project risk.

9 · Hostile questions — the non-negotiables

- **Is this just memory / caching / a DB?** → "Memory retrieves; our layer *gates*. The FSM refuses illegal transitions; the agent skips actions it already did. It changes decisions." (Only true once Steps 3–5 are wired.)
- **DDQN / RL / reward / training data?** → "There is no RL in the product. It was a design idea; I'm not shipping or claiming it." (Never imply otherwise.)
- **What did YOU code vs PentAGI?** → "The operational-state schema + FSM, the state-layer wiring, attack-path/findings visualization, web-pentest workflows, the UI. The base engine is PentAGI, MIT — I didn't rebuild a solved thing."
- **License violation?** → "No — MIT allows commercial derivatives; we retain attribution."
- **Feature, not a company?** → "The product is a pentester; the company is the audit/control layer for autonomous offense — compounding per-customer data + compliance."
- **What if the A/B shows nothing?** → "Then the thesis is wrong and I'll know by July 20 — I'd rather find out than sell a story."
- **Why you?** → origin story + background **[NEED G/H]**.

Two you must never bluff: RL/DDQN and metrics. One overclaim there and the whole application's credibility collapses. Honest "not built / not yet measured" is the winning answer.

10 · Execution plan → July 27

Wiring (Steps 0–6): 0) commit untracked worldstate pkg + migration behind a `WORLD_STATE_ENABLED` flag; 1) Store layer backing the tables + FSM; 2) `world_state_query` / `world_state_update` agent tools; 3) planner reads state digest before action; 4) executor writes entities + transitions after tool exec; 5) **pre-execution dedup gate** (the money demo); 6) credentials as typed entity + reuse. Prioritize 0–5.

A/B protocol: flag OFF (vanilla) vs ON; same target/model/prompt; ~5 runs × 2 targets × 2 conditions = 20 runs; targets = Juice Shop + DVWA (authorized only). Metrics: duplicate-action rate, tokens/finding, coverage, time-to-first-critical, cost/run, runtime. **Output:**

benchmarks/ab_results.csv committed — your single most valuable artifact.

Dates	Focus	Output	YC value
Jul 4	Commit worldstate + migration; add flag	Migration runs; flag toggles	Foundation
Jul 5–6	Store layer + tests	Tables become real	Real state
Jul 7–8	Tools + planner-reads	Agent queries state pre-action	"Query state" edge
Jul 9–10	Executor-writes + pre-exec gate	Duplicate scan skipped live	Demo core
Jul 11–12	Credentials entity + reuse	Cred stored & reused	Demo beat 2
Jul 13	Metrics scripts	benchmarks/metrics.py	Measurable claims
Jul 15	Cutline: stable? if not, invoke fallback	Go/no-go	Protects from fake data
Jul 15–17	Debug on Juice Shop + DVWA	Clean runs both flags	De-risk A/B
Jul 18–20	Full A/B, 20 runs	ab_results.csv + summary	#1 artifact
Jul 21	Record 90-sec demo	Demo video	Application field
Jul 22	Founder video (V1)	60-sec video	Required field
Jul 23–24	Draft full application (needs \$7)	Complete draft	Submission
Jul 25	Hostile-Q rehearsal	Confident answers	Interview readiness
Jul 26	Final edit; every claim → CSV/code	Fact-checked app	Kills credibility risk
Jul 27	Submit with half-day buffer	Submitted	Done

Cutline / fallback: if Steps 3–5 aren't stable by **July 15**, drop Step 6 + the formal A/B; ship the *live* demo (gate working on one target) with directional numbers from ~6 honest runs. Never let the deadline push you toward fabricated metrics.

11 · Final lists

Claims you must NOT make

Wired World State gating actions; planner-reads/executor-writes as done; any DDQN/RL/self-learning/policy; duplicate-prevention/credential-reuse/finding-dedup as shipped; any metric before the A/B; "we built an autonomous pentester" (it's PentAGI).

Strongest verified claims

(a) working, forked, MIT autonomous pentest platform running today; (b) designed + unit-tested lifecycle FSM and typed-entity/transition schema; (c) live attack-path + findings visualization; (d) a real, specific insight — vector memory ≠ operational model — no competitor's product addresses (confirmed vs PentAGI's own docs).

Top 3 rejection reasons → mitigations

1. **"Feature, not a company"** → wire Steps 3–6 so state changes decisions (demo it); lead with audit/compliance + per-customer-data moat, not the FSM.
2. **"You forked PentAGI — what's yours?"** → ship the wiring so your original code is the *working* state loop; disclose the fork proudly; quantify contribution with git.
3. **"No users, no metrics, no proof"** → land the A/B CSV (Jul 20) + one design partner or 3 real buyer conversations **[NEED F/D]**.

Submission checklist

- ☐ Application answers final (all [NEED] resolved)
- ☐ benchmarks/ab_results.csv committed + summary table
- ☐ 90-sec demo video (recorded + live take)
- ☐ Founder video V1 recorded
- ☐ Every claim maps to code or CSV (no orphan claims)
- ☐ Zero DDQN/RL/metric overclaims
- ☐ Fork disclosed; license/EULA/NOTICE consistent
- ☐ worldstate pkg + migration committed (not untracked)
- ☐ Backup name secured; domain checked
- ☐ Submitted with half-day buffer (not 11pm July 27)

Generated as a working strategy document for World Security. All technical claims verified against the codebase at `/Pentest_solution` ; unverifiable facts marked [NEED FROM YOU]. Competitor funding/batch figures marked [VERIFY] pending web confirmation.